

SERVICE ACTUEL · DATASPHERE-CLEYROP

Labels Datasphere, clearance et exports chiffrés

4 capacités analysées

services/datasphere-cleyrop · Python/FastAPI, SQLAlchemy/Alembic, binaire .NET Linux x86-64 propriétaire, Spark/JVM et S3.

Générer les labels, interroger un PDP de clearance et exporter les tables sous forme de fichiers chiffrés.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : IAM, Audit, Files, Query, Egress, Flows.

Périmètre de lecture

- Analyses existantes 11-ml-embeddings et 14-front-api reprises comme contexte ; vérifications ciblées des routes, outils et implémentations citées ci-dessous.
- Instructions racine et AGENTS.md du composant lorsqu'il existe ; aucun binaire propriétaire inspecté.
- Lecture statique ciblée pour finalisation ; les branches non exécutées et le déploiement restent non qualifiés.

Objets et cycle de vie

- Label policy/classification/category, représentation JSON/base64 ; requête de clearance ; EncryptJob et fichier chiffré.

Dépendances du service

- Binaire Datasphere non inspecté ; PDP clearance ; PostgreSQL ; Spark/Iceberg ; S3.

Fonctionnalités et compatibilité à préserver

DSPC-01 Cible non qualifiée

Construire et conserver un label de sensibilité

COMPORTEMENT OBSERVÉ

PUT /labels reçoit UUID dataset, policy, classification et category, retourne 202 et lance une BackgroundTask. Le handler appelle le binaire Datasphere pour générer un JSON, encode ce JSON en base64 puis upsert le label en base. Une erreur de tâche est journalisée ; le 202 ne constitue pas une preuve de génération terminée. Le binaire propriétaire n'a pas été inspecté.

UTILISATEURS ET RÔLES

Gestionnaires de labels et service data-model.

INTERFACES DANS LE CODE

PUT /labels ; BuildLabelCommandHandler ; DataspheresLabelServiceImpl.build_label.

DONNÉES ET ÉTATS

Label avec paramètres, JSON, base64 et dates PostgreSQL.

DÉPENDANCES ET ÉCHANGES

Binaire Datasphere .NET Linux x86-64, configuration/SPIF et base.

COMPORTEMENT CIBLE PROPOSÉ

IAM/Audit Rust contrôle la politique et un job durable génère un label versionné lié à une édition ; adaptateur propriétaire à remplacer ou qualifier explicitement.

MIGRATION DE CETTE CAPACITÉ

Importer labels et politiques, comparer byte/hash et décisions ; remplacer BackgroundTask par travail reprenable avec statut.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un 202 fournit un job consultable ; redémarrer après acceptation ne perd pas la génération ; paramètres identiques conservent le résultat attendu.

Références de l'analyse

[monorepo · services/datasphere-cleyrop/app/src/datasphere/application/router/label_router.py:43](#) 202 et tâche de fond.

[monorepo · services/datasphere-cleyrop/app/src/datasphere/domain/handler/build_label_command_handler.py:27](#) Création et upsert.

[monorepo · services/datasphere-cleyrop/app/src/datasphere/infra/service/label/datasphere_label_service_impl.py:31](#) Appel binaire et formats.

DSPC-02 **Cible non qualifiée****Relire le label et demander les décisions de clearance****COMPORTEMENT OBSERVÉ**

GET /labels trouve le label par UUID. POST /access/check reçoit des ressources et leurs sujets, valide les UUID, charge tous les labels et refuse les labels absents. Il construit pour chacun un metadata_binding base64 puis appelle le PDP /decide/by-id. L'adaptateur vérifie que la réponse est une liste de même cardinalité et la traduit en décisions par sujet ; réseau, TLS et retries sont configurables.

UTILISATEURS ET RÔLES

IAM, services consommateurs et utilisateurs soumis aux clearances.

INTERFACES DANS LE CODE

GET /labels?id=... ; POST /access/check.

DONNÉES ET ÉTATS

Labels stockés et décisions retournées, sans pouvoir de publication.

DÉPENDANCES ET ÉCHANGES

Clearance PDP HTTP/TLS et repository de labels.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust compose droits ordinaires et classification ; décisions versionnées avec politique, label et sujet, refus si indisponible selon contrat.

MIGRATION DE CETTE CAPACITÉ

Rejouer matrice politique/classification/sujets ; comparer réponses et erreurs avant changement de PDP ou binaire.

CRITÈRE D'ACCEPTATION À EXÉCUTER

UUID invalide et label absent ne donnent aucun ALLOW ; une réponse partielle du PDP est refusée, avec traçabilité de la version de politique.

Références de l'analyse

[monorepo · services/datasphere-cleyrop/app/src/datasphere/domain/handler/check_access_command_handler.py:32](#)

Construction des requêtes et validations.

[monorepo · services/datasphere-cleyrop/app/src/datasphere/infra/service/clearance/clearance_service_impl.py:151](#) Appel PDP et validation de réponse.

DSPC-03 **Cible non qualifiée****Extraire une table et produire un export chiffré ZTDF****COMPOTEMENT OBSERVÉ**

POST /labels/encrypt charge le label et télécharge la table par son tech_name via Spark. Le service écrit un seul fichier Parquet local en coalesçant, puis refuse un fichier vide ou supérieur à 2 Gio et le nettoie. Le binaire marque le fichier puis le chiffre en .ztdf ; le fichier est téléversé sur S3 et le répertoire temporaire nettoyé. Le code lit la table par nom sans pin d'édition explicite dans ce chemin.

UTILISATEURS ET RÔLES

Utilisateurs autorisés à exporter et opérateurs de données classifiées.

INTERFACES DANS LE CODE

POST /labels/encrypt ; SparkFileServiceImpl.download_file ; LabelService.encrypt.

DONNÉES ET ÉTATS

Parquet temporaire, label, artefact .ztdf et objet S3.

DÉPENDANCES ET ÉCHANGES

Spark/JVM, Iceberg accessible au catalogue Spark, S3 et binaire Datasphere.

COMPOTEMENT CIBLE PROPOSÉ

Query/Files Rust extrait une édition approuvée épinglée ; EXPORT reste un effet à reçu avec hash, politique et destinataire. Remplacer Spark et ses outils JVM.

MIGRATION DE CETTE CAPACITÉ

Qualifier lecteur Arrow/Parquet sans JVM et compatibilité ZTDF ; borner les flux avant matérialisation et conserver preuves de label/édition.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Exporter une édition donnée malgré une publication concurrente ; rejouer retrouve le même reçu ; aucun artefact privé n'est exposé avant contrôle de sortie.

Références de l'analyse

[monorepo · services/datasphere-cleyrop/app/src/datasphere/domain/handler/encrypt_command_handler.py:90](#) Limite de taille et mise en queue.

[monorepo · services/datasphere-cleyrop/app/src/datasphere/infra/service/file/spark_file_service_impl.py:250](#) Extraction Spark en un fichier.

[monorepo · services/datasphere-cleyrop/app/src/datasphere/infra/service/label/datasphere_label_service_impl.py:63](#) Marquage puis chiffrement.

DSPC-04 **Cible non qualifiée**

Ordonner les travaux de chiffrement et surveiller les dépendances

COMPORTEMENT OBSERVÉ

EncryptQueue est une `asyncio.Queue` en mémoire avec un worker démarré dans le `lifespan`. Le worker traite les jobs, journalise les exceptions puis marque la tâche terminée. Les instructions indiquent que les singletons Spark et queue sont créés au démarrage ; une panne d'initialisation Spark est seulement loggée et laisse l'encryption indisponible. Alembic est exécuté au démarrage ou via le mode `migrate`. Le 202 ne donne pas de reçu de livraison durable dans les routes lues.

UTILISATEURS ET RÔLES

SRE et opérateurs d'export.

INTERFACES DANS LE CODE

`EncryptQueue.start/stop/enqueue/_worker` ; `lifespan` et mode `migrate`.

DONNÉES ET ÉTATS

File de jobs locale au processus et fichiers temporaires.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, Spark, binaire et S3.

COMPORTEMENT CIBLE PROPOSÉ

Flows/Files Rust possède une queue durable, leases, limites de stockage et reçus d'effets ; `health` reflète chaque dépendance requise.

MIGRATION DE CETTE CAPACITÉ

Drainer les travaux connus, passer à une file persistée et tester reprises/cleanup avant retrait du runtime Spark.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Arrêt après extraction puis reprise n'abandonne pas de fichier sensible ; un Spark absent ou son remplaçant indisponible empêche une readiness d'export trompeuse.

Références de l'analyse

[monorepo · services/datasphere-cleyrop/app/src/datasphere/domain/handler/encrypt_command_handler.py:31](#) Queue mémoire, worker et erreurs.

[monorepo · services/datasphere-cleyrop/AGENTS.md:59](#) Comportement de démarrage et dépendances.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- IAM/Audit Rust contrôle la politique et un job durable génère un label versionné lié à une édition ; adaptateur propriétaire à remplacer ou qualifier explicitement.
- IAM Rust compose droits ordinaires et classification ; décisions versionnées avec politique, label et sujet, refus si indisponible selon contrat.
- Query/Files Rust extrait une édition approuvée épinglée ; EXPORT reste un effet à reçu avec hash, politique et destinataire. Remplacer Spark et ses outils JVM.

- Flows/Files Rust possède une queue durable, leases, limites de stockage et reçus d'effets ; health reflète chaque dépendance requise.

Plan de données proposé

- Lecture d'une édition autorisée puis production d'un export chiffré borné, avec reçu PostgreSQL ; aucune écriture directe Iceberg.

Découpage du core Rust

- label-registry, clearance-adapter, classified-export et durable-export-jobs.

Identité, autorisation et isolation

- Le label est un input de décision, pas une autorisation autonome d'export.
- La protection route/gateway et le mandat sur tech_name doivent être vérifiés de bout en bout ; non prouvés par les handlers seuls.

Exploitation et reprise

- Binaire testé seulement sur Linux ; tests d'intégration PDP exclus par défaut selon AGENTS, aucun lancé.
- File mémoire et BackgroundTasks ; migrations hors enforcement Alembic racine.

Séquence de migration

- Importer labels et politiques, comparer byte/hash et décisions ; remplacer BackgroundTask par travail reprenable avec statut.
- Rejouer matrice politique/classification/sujets ; comparer réponses et erreurs avant changement de PDP ou binaire.
- Qualifier lecteur Arrow/Parquet sans JVM et compatibilité ZTDF ; borner les flux avant matérialisation et conserver preuves de label/édition.
- Drainer les travaux connus, passer à une file persistée et tester reprises/cleanup avant retrait du runtime Spark.

Écarts et décisions encore ouverts

- Le contrat propriétaire ZTDF et le remplacement du runtime .NET doivent être décidés ; encapsulation temporaire non assimilée à core Rust.
- Le lien entre UUID label et tech_name de table doit être vérifié au contrôle d'entrée.

- Distinguer succès d'acceptation, de chiffrement, d'upload et remise au destinataire.

Références de l'analyse

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/application/router/label_router.py:43](#) 202 et tâche de fond.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/check_access_command_handler.py:32](#) Construction des requêtes et validations.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/encrypt_command_handler.py:90](#) Limite de taille et mise en queue.

[monorepo](#) · [services/datasphere-cleyrop/app/src/datasphere/domain/handler/encrypt_command_handler.py:31](#) Queue mémoire, worker et erreurs.